pedigrypst

A Typst package for building pedigrees, built on top of CeTZ

github.com/CrowdingFaun624/Pedigrypst

Version 0.1.0

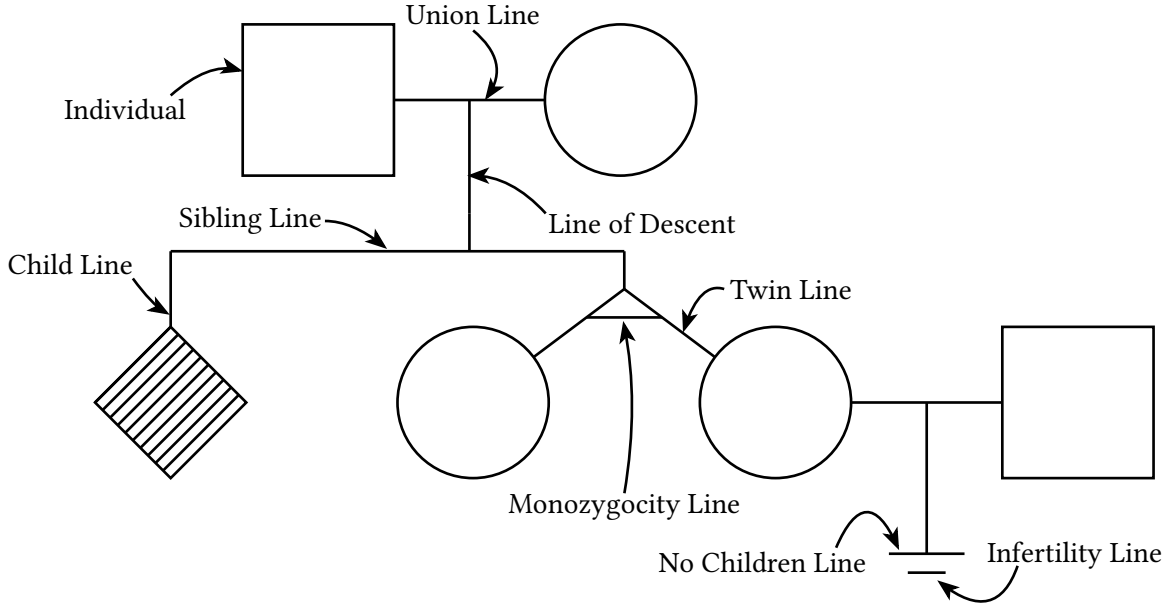
Guide

pedigrypst 1
- Pedigree 2
  - Individual 2
  - Duplicate 2
  - Union 2
  - Twin 2
  - Children 2
- Shapes 2
- Fills 3
- References 3
  - Individual References 3
  - Other References 3
- Decorations 4
  - Individual Decorations 4
  - Duplicate Decorations 4
  - Twin Decorations 4
  - Union Decorations 5
  - Children Decorations 5
  - Pedigree Decorations 5
- Example 6

Reference

Functions 7
- children 7
- duplicate 10
- individual 12
- pedigree 19
- twins 26
- union 28

Pedigree



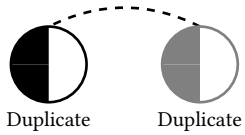
There are five different types of objects in a pedigree in Pedigrypst.

Individual

An **individual** is a single square, circle, or other shape that corresponds to a single organism.

Duplicate

A **duplicate** is a type of individual that mimics another individual. They act like individuals in all other ways.



Union

A **union** is a relationship between two individuals. They do not necessarily have any children.

Twin

A **twin** is an object that contains multiple individuals. If this object is the child of a children object, then they will be drawn like twins.

Children

A **children** is an object that references a union or an individual as its parents and multiple (or zero) individuals or twin objects as its children.

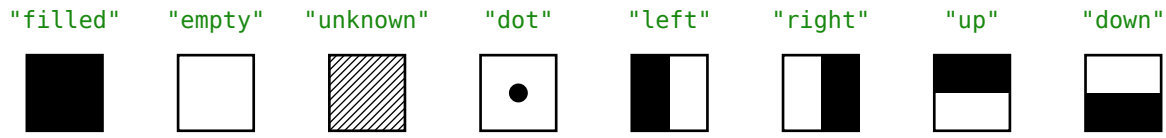
Shapes

There are four shapes/sexes that an individual can have, which is specified using the third parameter of `individual`.

The `"miscarriage"` shape is offset slightly higher than the other shapes.

Fills

There are eight preset fills available, which are specified using the `fill` parameter of `individual`



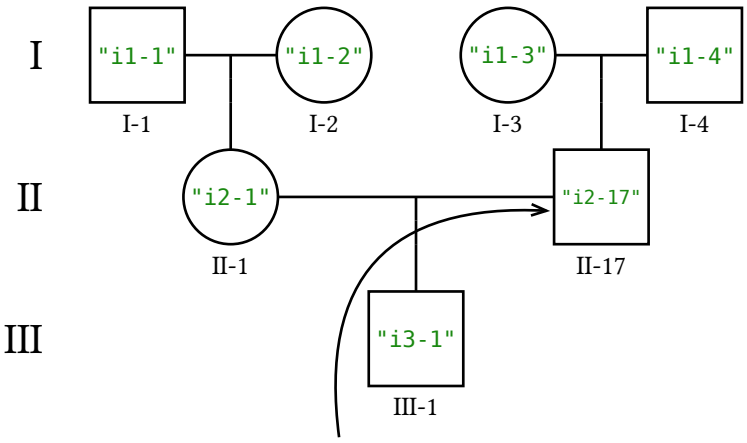
Additionally, quadrant notation may be used to specify the fill of each quadrant of a shape. This takes the form of a sequence of Roman numerals separated by hyphens. For example, the string `"I-III-IV"` encodes the fill of the individual shown to the right.

References

Objects in the pedigree can reference other objects using **references**.

Individual References

Individual references are strings that start with a lowercase `"i"`. Next is the one-indexed generation number of the individual. Then, there is a hyphen. Finally, there is the individual number, which is the number of the individual within its own generation.

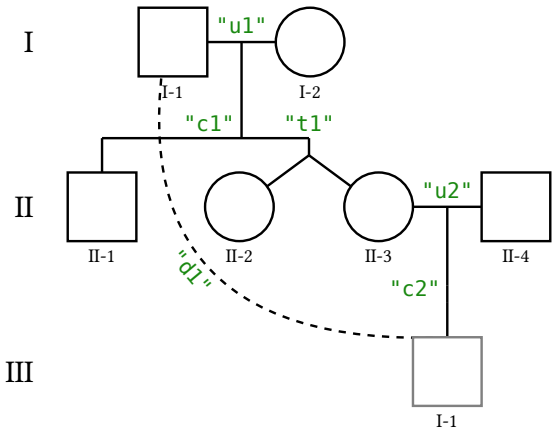


The individual number may be any number, as long as an individual has it. Notably, individual numbers may be skipped. Because individual II-17 is created using `individual(2, 17, "male")`, it can be referenced using `"i2-17"`.

Other References

Other objects can be referenced using their one-indexed ID. The ID of an object is how many objects of the same type, including itself, came before it.

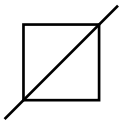
```
#pedigree({
  individual(1, 1, "male") // i1-1
  individual(1, 2, "female") // i1-2
  individual(2, 1, "male") // i2-1
  individual(2, 2, "female") // i2-2
  individual(2, 3, "female") // i2-3
  individual(2, 4, "male") // i2-4
  duplicate(3, "i1-1") // d1
  twins("i2-2", "i2-3") // t1
  union("i1-1", "i1-2") // u1
  union("i2-3", "i2-4") // u2
  children("u1", "i2-1", "t1") // c1
  children("u2", "d1") // c2
})
```



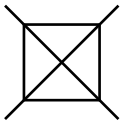
Duplicate are referenced with `"d"`, twins are referenced with `"t"`, unions are referenced with `"u"`, and childrens are referenced with `"c"`.

Decorations

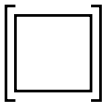
Individual Decorations



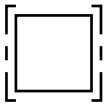
```
individual(
  1, 1, "male",
  dead: true,
)
```



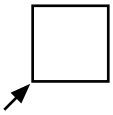
```
individual(
  1, 1, "male",
  dead: "double",
)
```



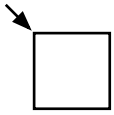
```
individual(
  1, 1, "male",
  adopted: true,
)
```



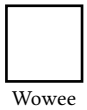
```
individual(
  1, 1, "male",
  adopted: "alt",
)
```



```
individual(
  1, 1, "male",
  propositus: true,
)
```



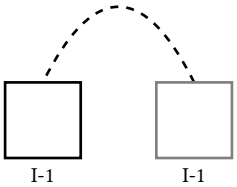
```
individual(
  1, 1, "male",
  propositus: top,
)
```



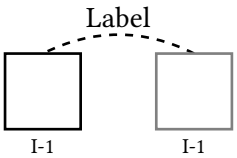

```
individual(
  1, 1, "male",
  label: [Wowee],
)
```

```
individual(
  1, 1, "male",
  label: [P],
)
```

Duplicate Decorations

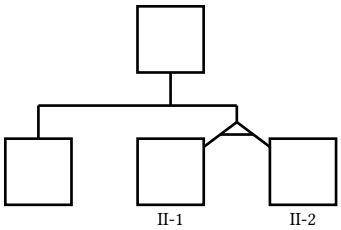



```
duplicate(1, "i1-1", bezier: 2)
```

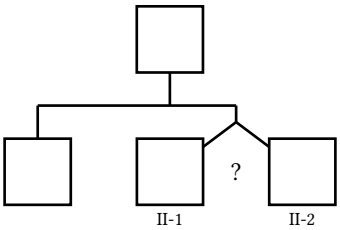



```
duplicate(1, "i1-1", label: [Label])
```

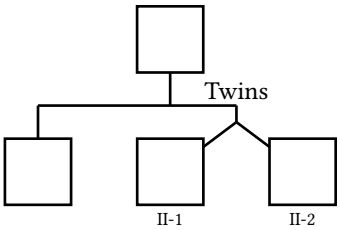
Twin Decorations




```
twins(
  "i2-1", "i2-2",
  monozygotic: true,
)
```

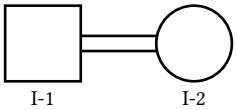



```
twins(
  "i2-1", "i2-2",
  monozygotic: "unknown",
)
```

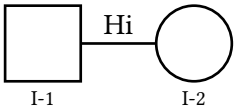



```
twins(
  "i2-1", "i2-2",
  label: [Twins],
)
```

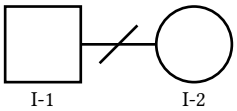
Union Decorations



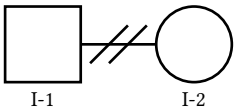

```
union(
  "i1-1", "i1-2",
  consanguineous: true,
)
```




```
union(
  "i1-1", "i1-2",
  label: [Hi],
)
```

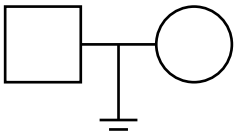



```
union(
  "i1-1", "i1-2",
  divorced: true,
)
```

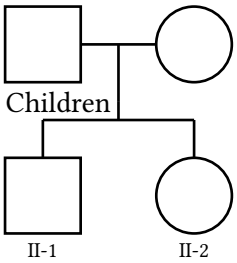



```
union(
  "i1-1", "i1-2",
  divorced: 2,
)
```

Children Decorations



```
children("u1", infertile: true)
```

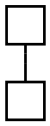



```
children(
  "u1", "i2-1", "i2-2",
  label: [Children],
)
```

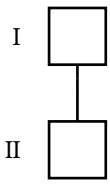
Pedigree Decorations

Note: The individuals and children are truncated from the code preview.

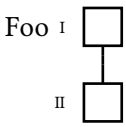
```
pedigree(generation-height: 0.5)
```



```
pedigree(generation-labels: false)
```




```
pedigree(length: 1.5cm)
```



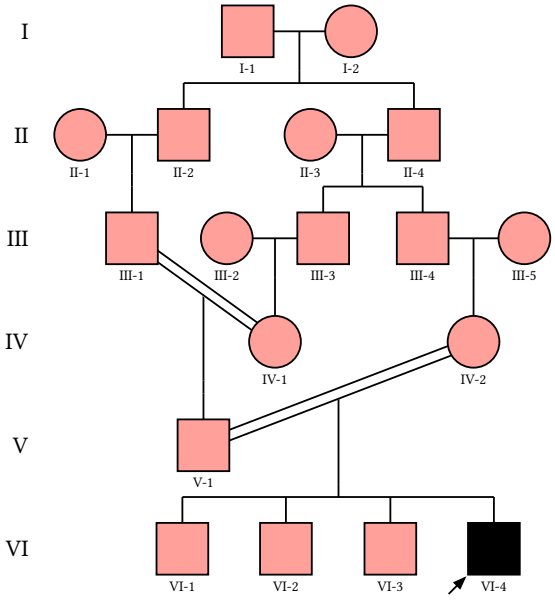

```
pedigree(draw: () => content(
  (-1, 0), [Foo],
))
```

Example

```
pedigree(
  length: 2cm,
  convergence-time: 250,
  empty-style: red.lighten(50%),
  {
    individual(1, 1, "male")
    individual(1, 2, "female")
    individual(2, 1, "female")
    individual(2, 2, "male")
    individual(2, 3, "female")
    individual(2, 4, "male")
    individual(3, 1, "male")
    individual(3, 2, "female")
    individual(3, 3, "male")
    individual(3, 4, "male")
    individual(3, 5, "female")
    individual(4, 1, "female")
    individual(4, 2, "female")
    individual(5, 1, "male")
    individual(6, 1, "male")
    individual(6, 2, "male")
    individual(6, 3, "male")
    individual(6, 4, "male", fill: "filled",
propositus: true)

    union("i1-1", "i1-2")
    union("i2-1", "i2-2")
    union("i2-3", "i2-4")
    union("i3-1", "i4-1", consanguineous:
true)
    union("i3-2", "i3-3")
    union("i3-4", "i3-5")
    union("i5-1", "i4-2", consanguineous:
true)

    children("u1", "i2-2", "i2-4")
    children("u2", "i3-1")
    children("u3", "i3-3", "i3-4")
    children("u4", "i5-1")
    children("u5", "i4-1")
    children("u6", "i4-2")
    children("u7", "i6-1", "i6-2", "i6-3",
"i6-4")
  }
)
```



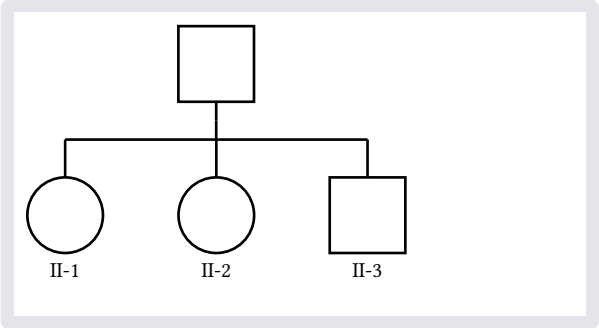
Functions

- children()
- duplicate()
- individual()
- pedigree()
- twins()
- union()

children

Creates a line of descent, a sibling line, and child lines from a union or individual to any number of children.

```
pedigree(
  {
    individual(1, 1, "male", label: none)
    individual(2, 1, "female")
    individual(2, 2, "female")
    individual(2, 3, "male")
    children("i1-1", "i2-1", "i2-2",
"i2-3")
  }
)
```



Parameters

```
children(
  parents: str,
  ..children: str,
  infertile: bool,
  label: content none,
  no-children-style: dictionary,
  line-of-descent-style: dictionary,
  sibling-line-style: dictionary,
  child-line-style: dictionary
)
```

parents `str`

A union reference, individual reference, or duplicate reference that specifies where the line of descent should be drawn from.

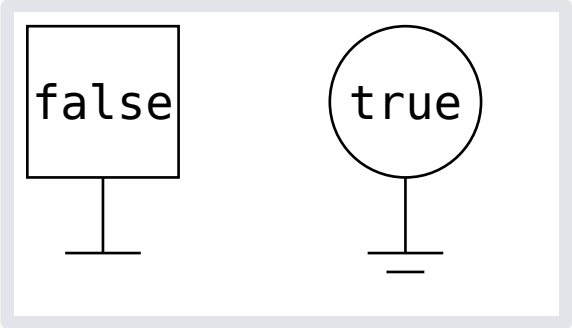
..children `str`

Multiple individual references, duplicate references, or twin references that specify what is a child of the parents. If empty, only the line of descent and a horizontal line indicating no children is drawn.

infertile `bool`

If no children are present, this specifies whether to draw an additional horizontal line beneath the no-children indicator.

```
pedigree(
  {
    individual(1, 1, "male", label:
none, in-label: `false`)
    individual(1, 2, "female", label:
none, in-label: `true`)
    children("i1-1", infertile: false)
    children("i1-2", infertile: true)
  }
)
```

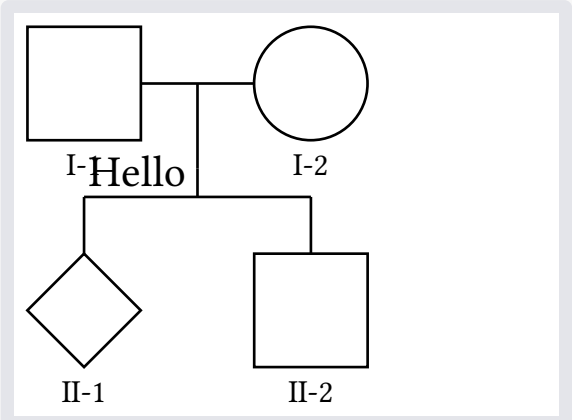


Default: `false`

label `content` or `none`

The content to put on the sibling line (if possible) next to the line of descent.

```
pedigree(
  {
    individual(1, 1, "male")
    individual(1, 2, "female")
    individual(2, 1, "unknown")
    individual(2, 2, "male")
    union("i1-1", "i1-2")
    children("u1", "i2-1", "i2-2",
label: [Hello])
  }
)
```

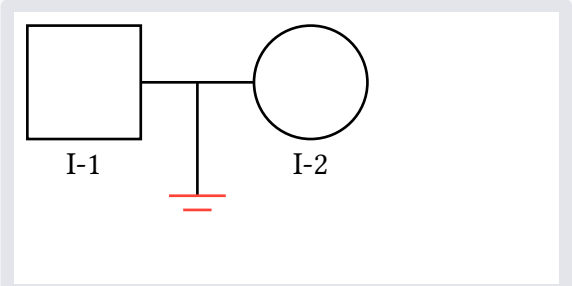


Default: `none`

no-children-style `dictionary`

The CeTZ style to give to the no children indicator for this children. Must be a dictionary.

```
pedigree(
  {
    individual(1, 1, "male")
    individual(1, 2, "female")
    union("i1-1", "i1-2")
    children("u1", infertile: true, no-
children-style: (paint: red))
  }
)
```

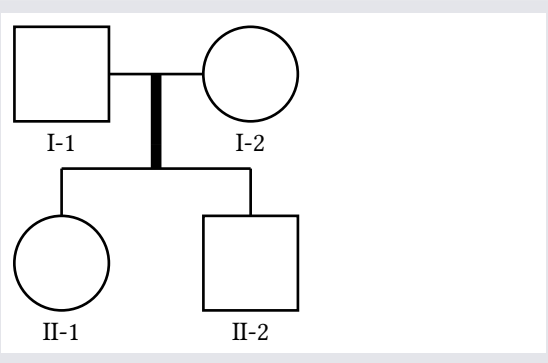


Default: `(:)`

line-of-descent-style `dictionary`

The CeTZ style to give to the line of descent. Must be a dictionary.

```
pedigree(
  {
    individual(1, 1, "male")
    individual(1, 2, "female")
    individual(2, 1, "female")
    individual(2, 2, "male")
    union("i1-1", "i1-2")
    children("u1", "i2-1", "i2-2", line-
of-descent-style: (thickness: 4pt))
  }
)
```

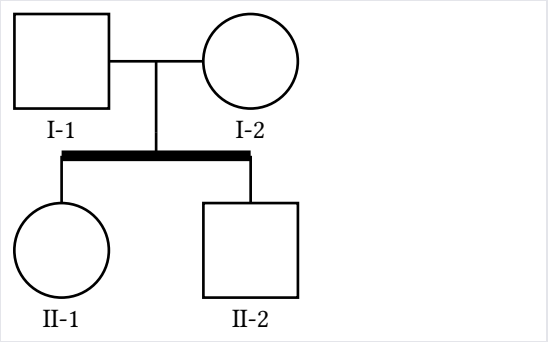


Default: `(:)`

sibling-line-style `dictionary`

The CeTZ style to give to the sibling line. Must be a dictionary.

```
pedigree(
  {
    individual(1, 1, "male")
    individual(1, 2, "female")
    individual(2, 1, "female")
    individual(2, 2, "male")
    union("i1-1", "i1-2")
    children("u1", "i2-1", "i2-2",
sibling-line-style: (thickness: 4pt))
  }
)
```

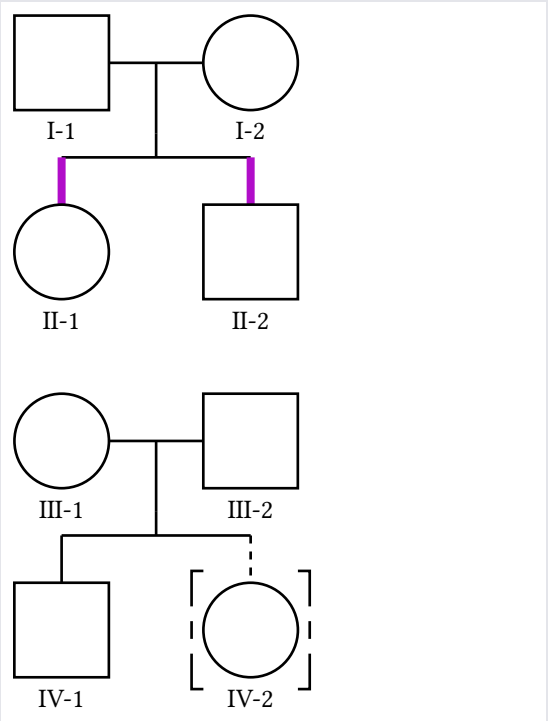


Default: `(:)`

child-line-style `dictionary`

The CeTZ style to give to the child line. The styles must be dictionaries. If a dictionary is given, the style will be applied to all child lines. If an array is given with the same length as the number of children, each style will be applied to the corresponding child line.

```
pedigree(
  {
    individual(1, 1, "male")
    individual(1, 2, "female")
    individual(2, 1, "female")
    individual(2, 2, "male")
    individual(3, 1, "female")
    individual(3, 2, "male")
    individual(4, 1, "male")
    individual(4, 2, "female", adopted:
"alt")
    union("i1-1", "i1-2")
    union("i3-1", "i3-2")
    children("u1", "i2-1", "i2-2",
child-line-style: (paint: purple,
thickness: 3pt))
    children("u2", "i4-1", "i4-2",
child-line-style: ((:), (dash:
"dashed")))
  }
)
```

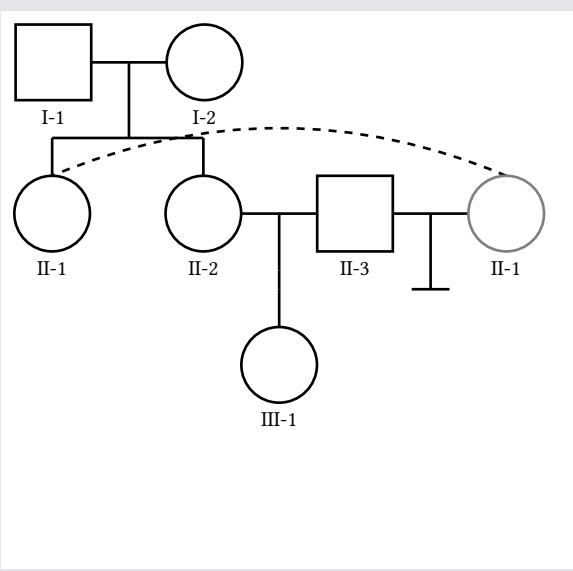


Default: `(:)`

duplicate

Creates a lighter copy of an individual and shows its link to it using a curved line.

```
pedigree(
  {
    individual(1, 1, "male")
    individual(1, 2, "female")
    individual(2, 1, "female")
    individual(2, 2, "female")
    individual(2, 3, "male")
    duplicate(2, "i2-1", bezier: 0.42)
    // right here ^
    individual(3, 1, "female")
    union("i1-1", "i1-2")
    union("i2-2", "i2-3")
    union("i2-3", "d1")
    children("u1", "i2-1", "i2-2")
    children("u2", "i3-1")
    children("u3")
  }
)
```



Parameters

```
duplicate(
  generation: int,
  individual: str,
  bezier: int float none,
  label: content none,
  curve-style: dictionary,
  lightness: ratio
)
```

generation `int`

The generation to place the individual in.

individual `str`

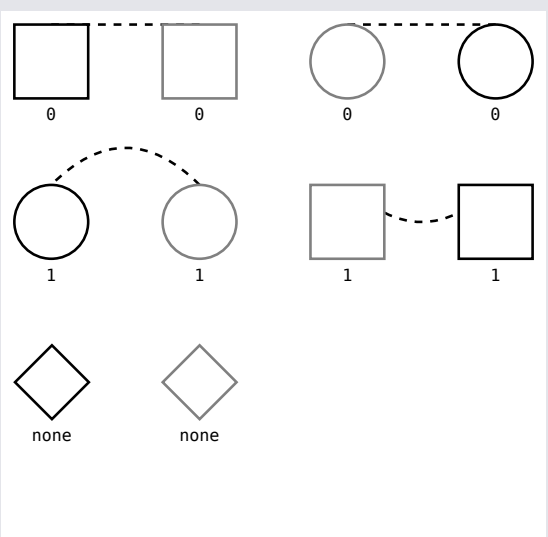
The individual reference to the individual this duplicate is duplicating.

bezier `int` or `float` or `none`

Determines the control point of the bezier for the curved line. If `none`, the curved line will not be drawn.

If the duplicate is to the right of the duplicated individual, positive values will curve the line upwards. If the duplicate is to the left of the duplicated individual, positive values will curve the line downwards.

```
pedigree(
  {
    individual(1, 1, "male", label: `0`)
    duplicate(1, "i1-1", bezier: 0)
    duplicate(1, "i1-4", bezier: 0)
    individual(1, 4, "female", label:
`0`)
    individual(2, 1, "female", label:
`1`)
    duplicate(2, "i2-1", bezier: 1)
    duplicate(2, "i2-4", bezier: 1)
    individual(2, 4, "male", label: `1`)
    individual(3, 1, "unknown", label:
`none`)
    duplicate(3, "i3-1", bezier: none)
  }
)
```

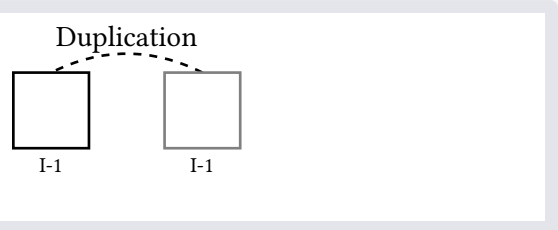


Default: `0.5`

label `content` or `none`

The content to place next to the curve.

```
pedigree(
  {
    individual(1, 1, "male")
    duplicate(1, "i1-1", label:
[Duplication])
  }
)
```

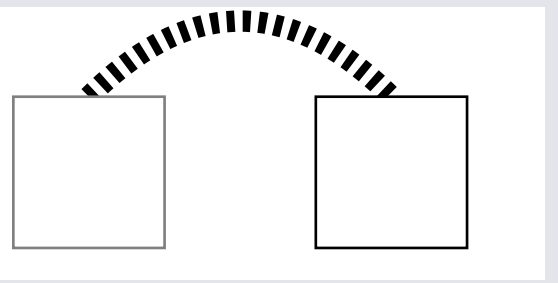


Default: `none`

curve-style `dictionary`

The CeTZ style to give to the curved line. Must be a dictionary. Defaults to being dashed.

```
pedigree(
  {
    duplicate(1, "i1-1", bezier: -1,
curve-style: (thickness: 8pt))
    individual(1, 1, "male", label:
none)
  }
)
```

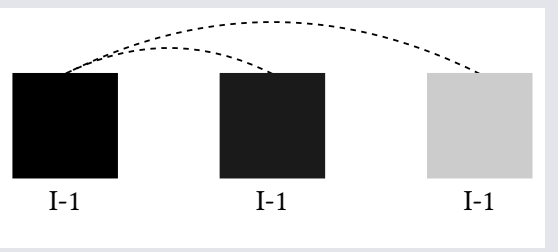


Default: `(:)`

lightness `ratio`

The amount to brighten the duplicate by.

```
pedigree(
  {
    individual(1, 1, "male", fill:
"filled")
    duplicate(1, "i1-1", lightness: 10%)
    duplicate(1, "i1-1", lightness: 80%)
  }
)
```



Default: `50%`

individual

Creates a single individual in a pedigree.

Parameters

```
individual(
  generation: int,
  ind-number: int,
  sex: str,
  fill: str,
  dead: bool str,
  adopted: bool str,
  propositus: bool alignment,
  label: content none auto,
  in-label: content none,
  stroke-style: dictionary,
  empty-style: color none auto,
  fill-style: color none auto,
  dead-style: dictionary,
  adopted-style: dictionary,
  propositus-style: dictionary
)
```

generation `int`

The generation of the individual, starting from 1.

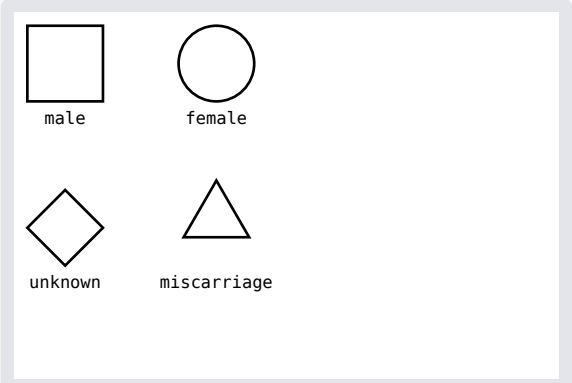
ind-number `int`

The ordinality of the individual within its generation, starting from 1. This is used only for the label and to reference this individual.

sex `str`

The shape of the individual. Must be `"male"`, `"female"`, `"unknown"`, or `"miscarriage"`.

```
pedigree(
  {
    individual(1, 1, "male", label: `male`)
    individual(1, 2, "female", label: `female`)
    individual(2, 1, "unknown", label: `unknown`)
    individual(2, 2, "miscarriage", label: `miscarriage`)
  }
)
```

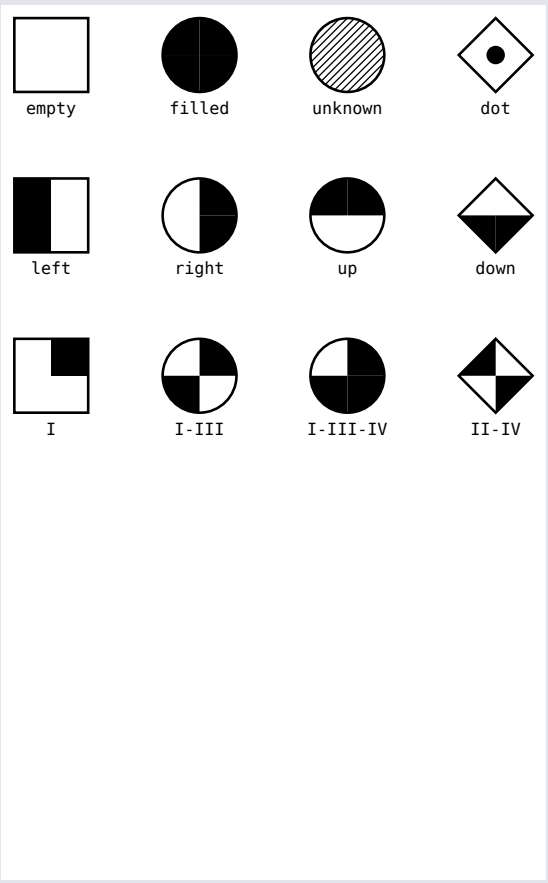


fill `str`

The pattern of filling the individual. Must be `"empty"`, `"filled"`, `"unknown"`, `"dot"`, `"left"`, `"right"`, `"up"`, `"down"`, or use quadrant notation.

Quadrant notation involves placing the capitalized Roman numerals of a quadrant, in order, separated by hyphens. For example, `"I-III-IV"` is valid quadrant notation.

```
pedigree(
  {
    individual(1, 1, "male", fill: "empty", label: `empty`)
    individual(1, 2, "female", fill: "filled", label: `filled`)
    individual(1, 3, "female", fill: "unknown", label: `unknown`)
    individual(1, 4, "unknown", fill: "dot", label: `dot`)
    individual(2, 1, "male", fill: "left", label: `left`)
    individual(2, 2, "female", fill: "right", label: `right`)
    individual(2, 3, "female", fill: "up", label: `up`)
    individual(2, 4, "unknown", fill: "down", label: `down`)
    individual(3, 1, "male", fill: "I", label: `I`)
    individual(3, 2, "female", fill: "I-III", label: `I-III`)
    individual(3, 3, "female", fill: "I-III-IV", label: `I-III-IV`)
    individual(3, 4, "unknown", fill: "II-IV", label: `II-IV`)
  }
)
```



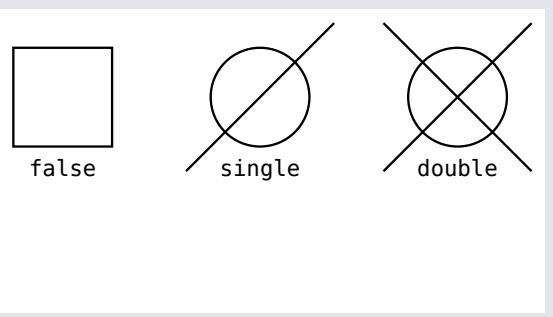
Default: `"empty"`

dead `bool` or `str`

Whether to draw the deceased indicator on the individual. Can be `true`, `false`, `"true"`, `"false"`, `"single"`, and `"double"`.

`true`, `"true"`, and `"single"` all create one cross up and to the right, and `"double"` creates another cross up and to the left.

```
pedigree(
  {
    individual(1, 1, "male", dead:
false, label: `false`)
    individual(1, 2, "female", dead:
"single", label: `single`)
    individual(1, 3, "female", dead:
"double", label: `double`)
  }
)
```



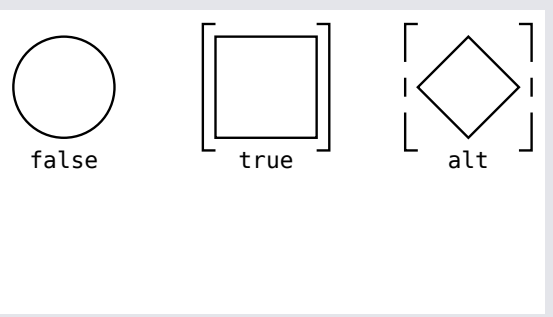
Default: `false`

adopted `bool` or `str`

Whether to draw the adopted brackets around the individual. Can be `true`, `false`, `"true"`, `"false"`, and `"alt"`.

`true` and `"true"` draw whole brackets around the individual, while `"alt"` draws brackets with two slits near the middle.

```
pedigree(
  {
    individual(1, 1, "female", adopted:
false, label: `false`)
    individual(1, 2, "male", adopted:
true, label: `true`)
    individual(1, 3, "unknown", adopted:
"alt", label: `alt`)
  }
)
```

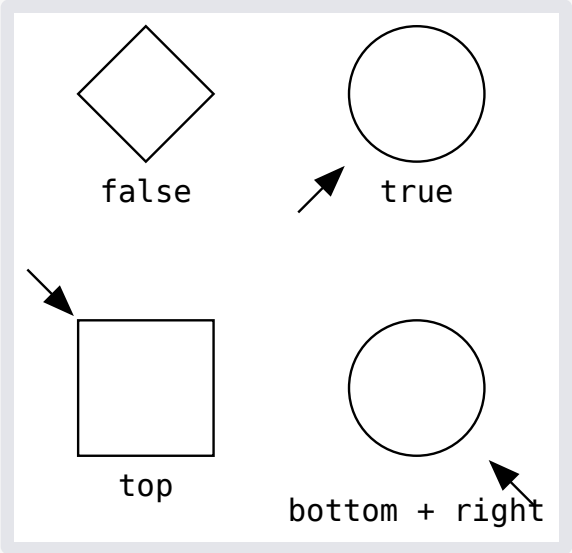


Default: `false`

propositus `bool` or `alignment`

Whether to draw a propositus. If the alignment is unspecified, the propositus will go in the bottom-left corner.

```
pedigree(
  {
    individual(1, 1, "unknown",
propositus: false, label: `false`)
    individual(1, 2, "female",
propositus: true, label: `true`)
    individual(2, 1, "male", propositus:
top, label: `top`)
    individual(2, 2, "female",
propositus: bottom + right, label:
`bottom + right`)
  }
)
```

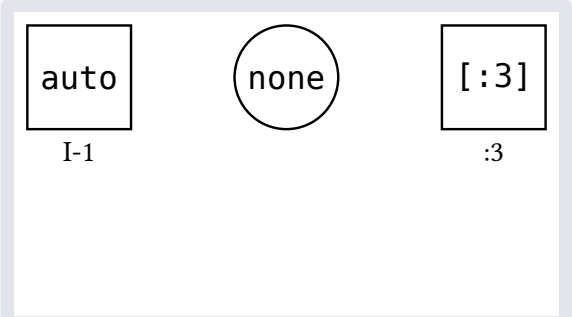


Default: `false`

label `content` or `none` or `auto`

The label to place beneath the individual. If auto, the individual's generation and ind-number will be shown.

```
pedigree(
  {
    individual(1, 1, "male", label:
auto, in-label: `auto`)
    individual(1, 2, "female", label:
none, in-label: `none`)
    individual(1, 3, "male", label:
[:3], in-label: `[:3]`)
  }
)
```

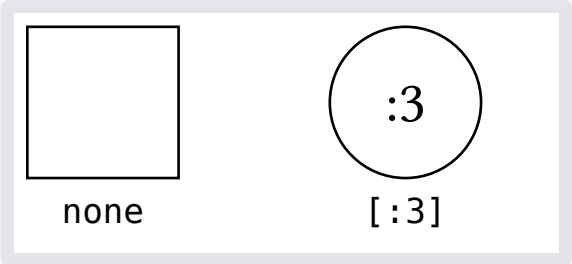


Default: `auto`

in-label `content` or `none`

Content to put inside of the individual.

```
pedigree(
  {
    individual(1, 1, "male", in-label:
none, label: `none`)
    individual(1, 2, "female", in-label:
[:3], label: `[:3]`)
  }
)
```



Default: `none`

stroke-style `dictionary`

The CeTZ style to give to the outline of the individual. Must be a dictionary.

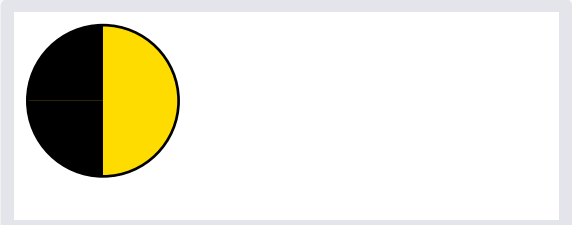
```
pedigree(
  {
    individual(1, 1, "male", label:
none, stroke-style: (paint: blue, dash:
"dashed"))
  }
)
```

Default: `(:)`

empty-style `color` or `none` or `auto`

The color to give to the empty part of all individuals.

```
pedigree(
  {
    individual(1, 1, "female", fill:
"left", label: none, empty-style:
yellow)
  }
)
```

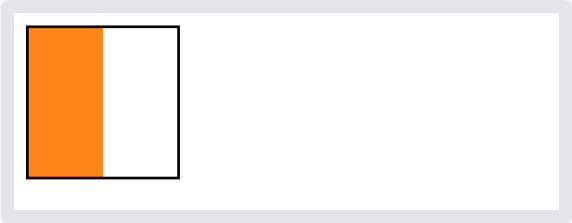


Default: `auto`

fill-style `color` or `none` or `auto`

The color to give to the filled part of all individuals.

```
pedigree(
  {
    individual(1, 1, "male", fill:
"left", label: none, fill-style: orange)
  }
)
```

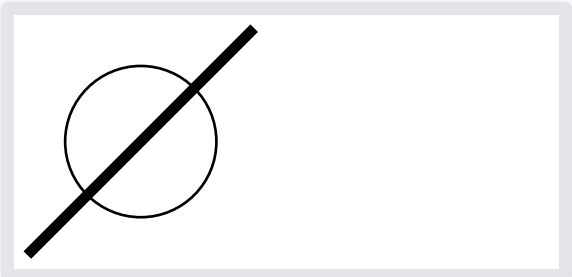


Default: `auto`

dead-style `dictionary`

The CeTZ style to give to all deceased crosses. Must be a dictionary.

```
pedigree(
  {
    individual(1, 1, "female", dead:
true, label: none, dead-style:
(thickness: 4pt))
  }
)
```

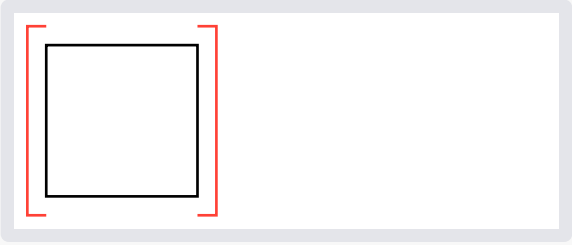


Default: `(:)`

adopted-style `dictionary`

The CeTZ style to give to all adoption brackets. Must be a dictionary.

```
pedigree(
  {
    individual(1, 1, "male", adopted:
true, label: none, adopted-style:
(paint: red))
  }
)
```

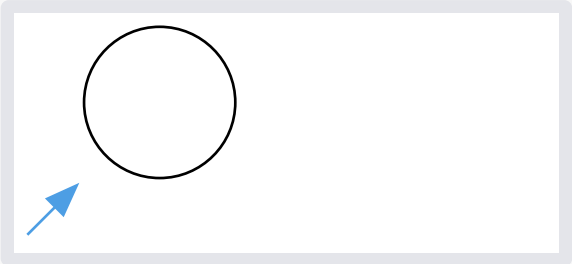


Default: `(:)`

propositus-style `dictionary`

The CeTZ style to give to all propositi. Must be a dictionary.

```
pedigree(
  {
    individual(1, 1, "female",
propositus: true, label: none,
propositus-style: (paint:
blue.lighten(30%)))
  }
)
```



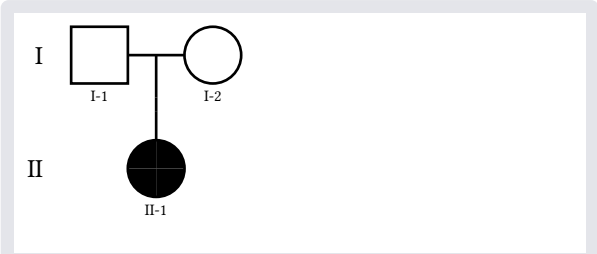
Default: `(:)`

pedigree

Creates a pedigree.

Use syntax such as:

```
pedigree(length: 1.5cm, {
  individual(1, 1, "male")
  individual(1, 2, "female")
  individual(2, 1, "female", fill:
"filled")
  union("i1-1", "i1-2")
  children("u1", "i2-1")
})
```



Parameters

```
pedigree(
  contents,
  convergence-time: int,
  generation-height: int float,
  length: length,
  generation-labels: bool,
  draw: function none,
  stroke-style: dictionary,
  empty-style: color none auto,
  fill-style: color none auto,
  dead-style: dictionary,
  adopted-style: dictionary,
  propositus-style: dictionary,
  duplicate-curve-style: dictionary,
  twin-style: dictionary,
  monozygotic-style: dictionary,
  union-style: dictionary,
  divorced-style: dictionary,
  no-children-style: dictionary,
  line-of-descent-style: dictionary,
  sibling-line-style: dictionary,
  child-line-style: dictionary
) -> content
```

contents

The individuals, duplicates, twins, unions, and childrens objects to display in the pedigree.

convergence-time `int`

The number of steps to use in the force-directed graph drawing. For larger pedigrees, this may need to be several hundred.

Default: `100`

generation-height `int` or `float`

The visual minimum height of each generation.

Default: `1`

length `length`

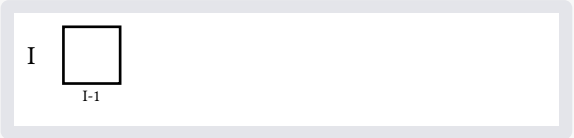
The length used on the CeTZ canvas. Scales the pedigree relative to `1cm`.

Default: `1cm`

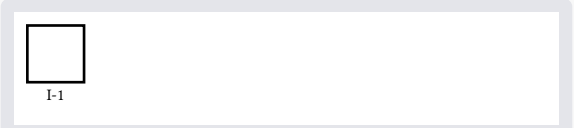
generation-labels `bool`

Whether to add Roman numerals indicating generation number to the left side of the pedigree.

```
pedigree(generation-labels: true,
length: 1.5cm, {
  individual(1, 1, "male")
})
```




```
pedigree(generation-labels: false,
length: 1.5cm, {
  individual(1, 1, "male")
})
```



Default: `true`

draw `function` or `none`

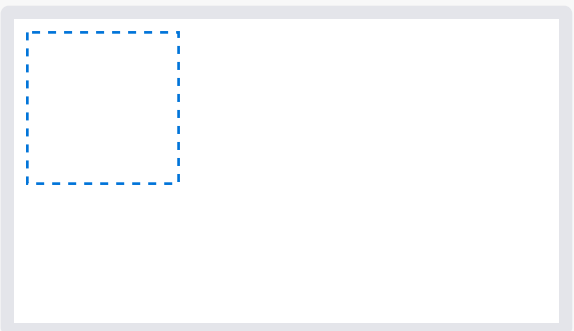
Returns CeTZ content to add to the canvas after the pedigree is drawn.

Default: `none`

stroke-style `dictionary`

The CeTZ style to give to the outline of all individuals. Must be a dictionary.

```
pedigree(
  stroke-style: (
    paint: blue,
    dash: "dashed",
  ),
  {
    individual(1, 1, "male", label:
none)
  }
)
```

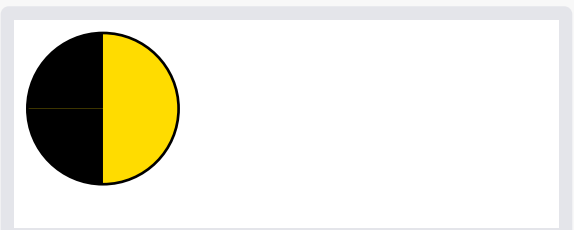


Default: `(:)`

empty-style `color` or `none` or `auto`

The color to give to the empty part of all individuals.

```
pedigree(
  empty-style: yellow,
  {
    individual(1, 1, "female", fill:
"left", label: none)
  }
)
```



Default: `auto`

fill-style `color` or `none` or `auto`

The color to give to the filled part of all individuals.

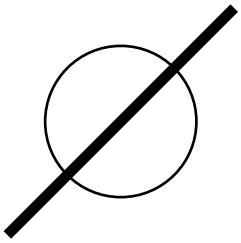
```
pedigree(
  fill-style: orange,
  {
    individual(1, 1, "male", fill:
"left", label: none)
  }
)
```

Default: `auto`

dead-style `dictionary`

The CeTZ style to give to all deceased crosses. Must be a dictionary.

```
pedigree(
  dead-style: (thickness: 4pt),
  {
    individual(1, 1, "female", dead:
true, label: none)
  }
)
```

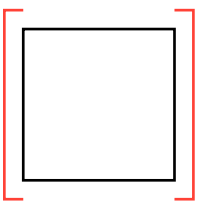


Default: `(:)`

adopted-style `dictionary`

The CeTZ style to give to all adoption brackets. Must be a dictionary.

```
pedigree(
  adopted-style: (paint: red),
  {
    individual(1, 1, "male", adopted:
true, label: none)
  }
)
```

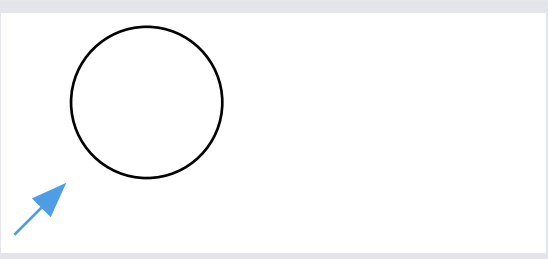


Default: `(:)`

propositus-style `dictionary`

The CeTZ style to give to all propositi. Must be a dictionary.

```
pedigree(
  propositus-style: (paint:
blue.lighten(30%)),
  {
    individual(1, 1, "female",
propositus: true, label: none)
  }
)
```

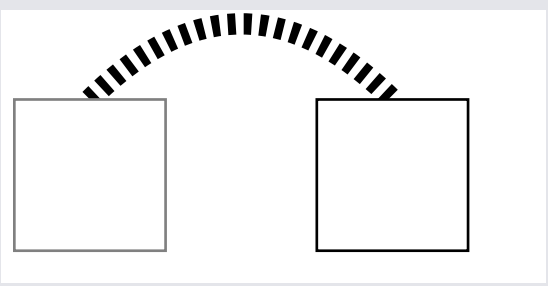


Default: `(:)`

duplicate-curve-style `dictionary`

The CeTZ style to give to all duplicate lines. Must be a dictionary. Defaults to being dashed.

```
pedigree(
  duplicate-curve-style: (thickness:
8pt),
  {
    duplicate(1, "i1-1", bezier: -1)
    individual(1, 1, "male", label:
none)
  }
)
```

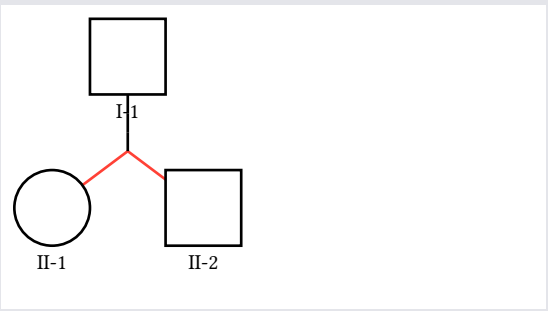


Default: `(:)`

twin-style `dictionary`

The CeTZ style to give to all twin lines. Must be a dictionary.

```
pedigree(
  twin-style: (paint: red),
  {
    individual(1, 1, "male")
    individual(2, 1, "female")
    individual(2, 2, "male")
    twins("i2-1", "i2-2")
    children("i1-1", "t1")
  }
)
```

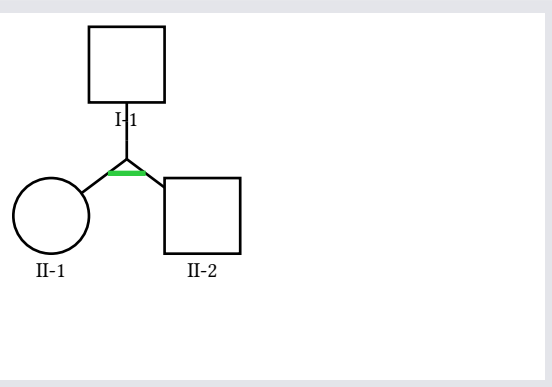


Default: `(:)`

monozygotic-style `dictionary`

The CeTZ style to give to the monozygocity indicator of all twins. Must be a dictionary.

```
pedigree(
  monozygotic-style: (paint: green,
thickness: 2pt),
  {
    individual(1, 1, "male")
    individual(2, 1, "female")
    individual(2, 2, "male")
    twins("i2-1", "i2-2", monozygotic:
true)
    children("i1-1", "t1")
  }
)
```

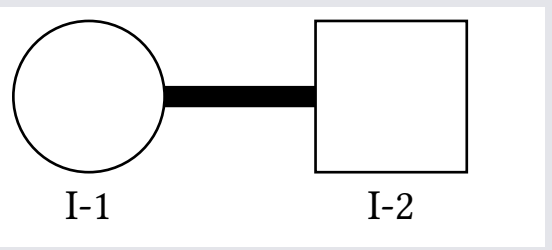


Default: `(:)`

union-style `dictionary`

The CeTZ style to give to the union line of all unions. Must be a dictionary.

```
pedigree(
  union-style: (thickness: 8pt),
  {
    individual(1, 1, "female")
    individual(1, 2, "male")
    union("i1-1", "i1-2")
  }
)
```

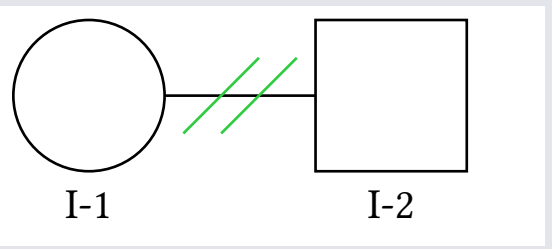


Default: `(:)`

divorced-style `dictionary`

The CeTZ style to give to the separation indicators of all unions. Must be a dictionary.

```
pedigree(
  divorced-style: (paint: green),
  {
    individual(1, 1, "female")
    individual(1, 2, "male")
    union("i1-1", "i1-2", divorced: 2)
  }
)
```

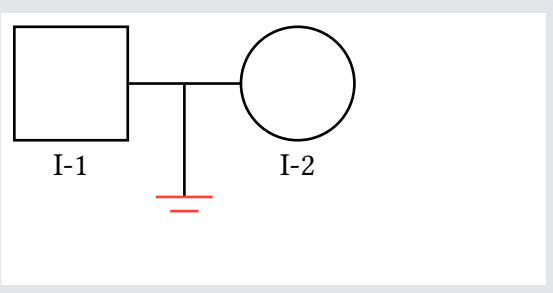


Default: `(:)`

no-children-style `dictionary`

The CeTZ style to give to the no children indicator for all children. Must be a dictionary.

```
pedigree(
  no-children-style: (paint: red),
  {
    individual(1, 1, "male")
    individual(1, 2, "female")
    union("i1-1", "i1-2")
    children("u1", infertile: true)
  }
)
```

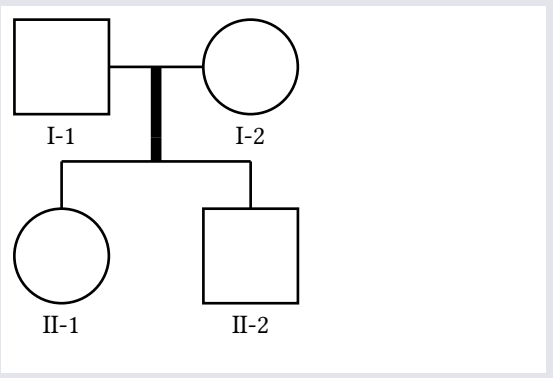


Default: `(:)`

line-of-descent-style `dictionary`

The CeTZ style to give to all lines of descent. Must be a dictionary.

```
pedigree(
  line-of-descent-style: (thickness:
4pt),
  {
    individual(1, 1, "male")
    individual(1, 2, "female")
    individual(2, 1, "female")
    individual(2, 2, "male")
    union("i1-1", "i1-2")
    children("u1", "i2-1", "i2-2")
  }
)
```

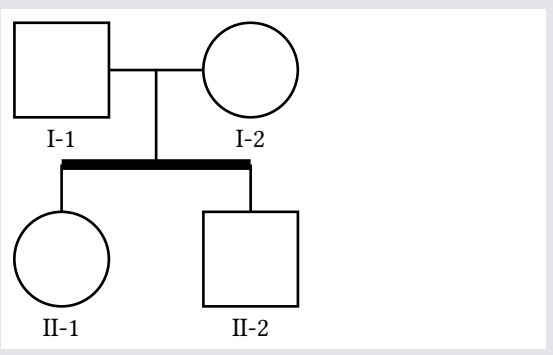


Default: `(:)`

sibling-line-style `dictionary`

The CeTZ style to give to all sibling lines. Must be a dictionary.

```
pedigree(
  sibling-line-style: (thickness: 4pt),
  {
    individual(1, 1, "male")
    individual(1, 2, "female")
    individual(2, 1, "female")
    individual(2, 2, "male")
    union("i1-1", "i1-2")
    children("u1", "i2-1", "i2-2")
  }
)
```

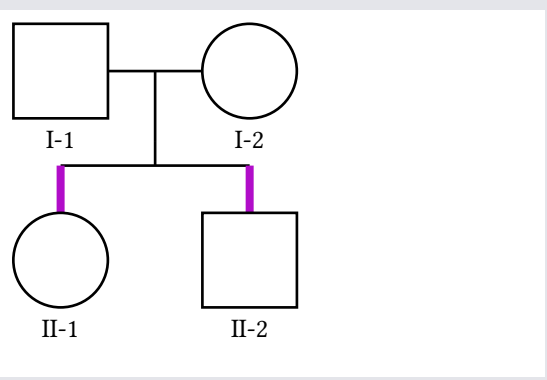


Default: `(:)`

child-line-style `dictionary`

The CeTZ style to give to all child lines. Must be a dictionary.

```
pedigree(
  child-line-style: (paint: purple,
thickness: 3pt),
  {
    individual(1, 1, "male")
    individual(1, 2, "female")
    individual(2, 1, "female")
    individual(2, 2, "male")
    union("i1-1", "i1-2")
    children("u1", "i2-1", "i2-2")
  }
)
```

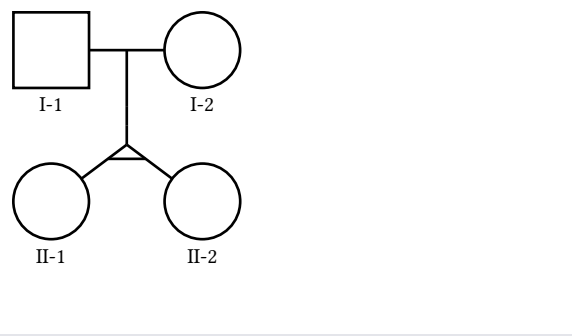


Default: `(:)`

twins

Creates twins

```
pedigree(
  {
    individual(1, 1, "male")
    individual(1, 2, "female")
    individual(2, 1, "female")
    individual(2, 2, "female")
    twins("i2-1", "i2-2", monozygotic:
true)
    union("i1-1", "i1-2")
    children("u1", "t1")
})
```



Parameters

```
twins(
  ..individuals: str,
  monozygotic: bool str array,
  label: content none,
  style: dictionary,
  monozygotic-style: dictionary
)
```

..individuals `str`

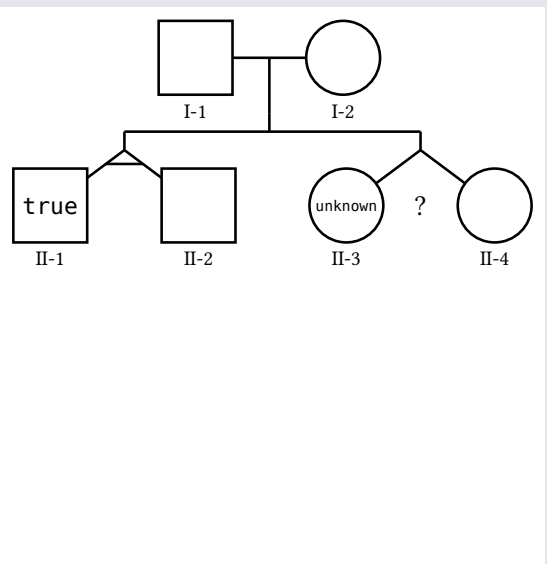
Multiple individual or duplicate references to place in the twins. Can have more than two items.

monozygotic `bool` or `str` or `array`

Whether to draw the monozygocity indicator between each pair of twins. If not an array, it is converted into an array of length `1`. The length of the array should be one less than the number of twins.

Each item of the array can be `true`, `false`, `"true"`, `"false"`, or `"unknown"`. If `true` or `"true"`, a horizontal line will be drawn near the sibling line. If `"unknown"`, a question mark will be drawn between the individuals.

```
pedigree(
  {
    individual(1, 1, "male")
    individual(1, 2, "female")
    individual(2, 1, "male", in-label:
`true`)
    individual(2, 2, "male")
    individual(2, 3, "female", in-label:
text(`unknown`, size: 7pt))
    individual(2, 4, "female")
    twins("i2-1", "i2-2", monozygotic:
(true,))
    twins("i2-3", "i2-4", monozygotic:
("unknown",))
    union("i1-1", "i1-2")
    children("u1", "t1", "t2")
  }
)
```

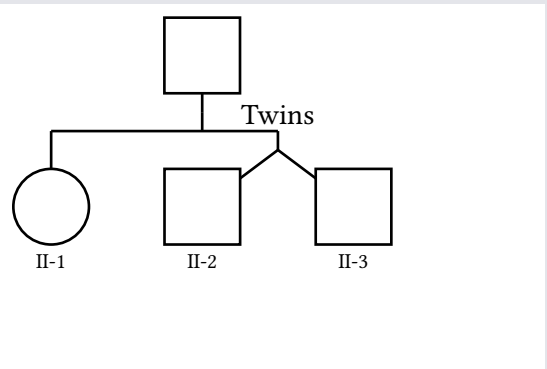


Default: `false`

label `content` or `none`

The label to place on the sibling line above the center of the twins.

```
pedigree(
  {
    individual(1, 1, "male", label:
none)
    individual(2, 1, "female")
    individual(2, 2, "male")
    individual(2, 3, "male")
    twins("i2-2", "i2-3", label:
[Twins])
    children("i1-1", "i2-1", "t1")
  }
)
```

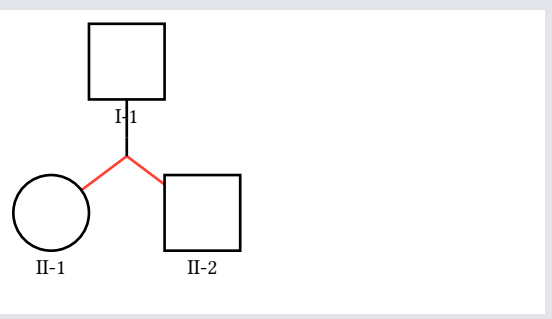


Default: `none`

style `dictionary`

The CeTZ style to draw all of the twin lines for these twins with. Must be a dictionary.

```
pedigree(
  {
    individual(1, 1, "male")
    individual(2, 1, "female")
    individual(2, 2, "male")
    twins("i2-1", "i2-2", style: (paint:
red))
    children("i1-1", "t1")
  }
)
```

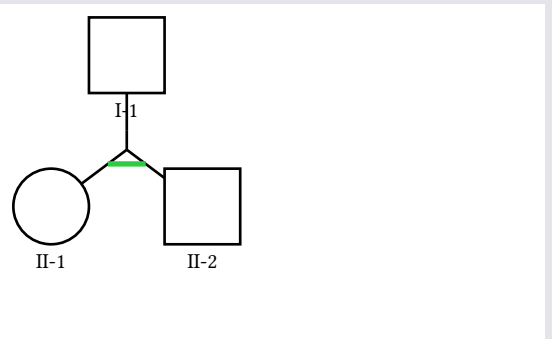


Default: `(:)`

monozygotic-style `dictionary`

The CeTZ style to draw the monozygocity indicators for these twins with. Must be a dictionary.

```
pedigree(
  {
    individual(1, 1, "male")
    individual(2, 1, "female")
    individual(2, 2, "male")
    twins("i2-1", "i2-2", monozygotic:
true, monozygotic-style: (paint: green,
thickness: 2pt))
    children("i1-1", "t1")
  }
)
```

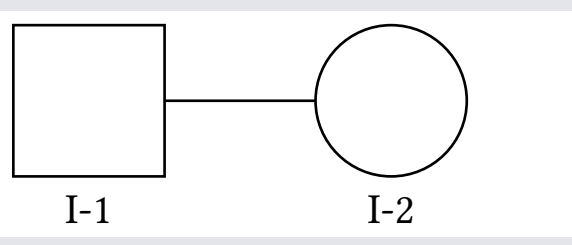


Default: `(:)`

union

Creates a union between two individuals. It does not specify the children, only the two individuals.

```
pedigree(
  {
    individual(1, 1, "male")
    individual(1, 2, "female")
    union("i1-1", "i1-2")
  }
)
```



Parameters

```
union(
  individual-1: str,
  individual-2: str,
  consanguineous: bool,
  divorced: bool int,
  label: content none,
  style: dictionary,
  divorced-style: dictionary
)
```

individual-1 `str`

The first individual reference of the union.

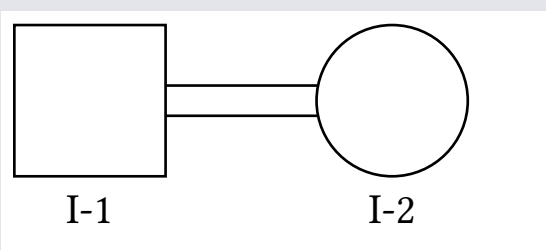
individual-2 `str`

The second individual reference of the union.

consanguineous `bool`

Whether to draw the union with double lines, indicating a consanguineous relationship.

```
pedigree(
  {
    individual(1, 1, "male")
    individual(1, 2, "female")
    union("i1-1", "i1-2",
consanguineous: true)
  }
)
```



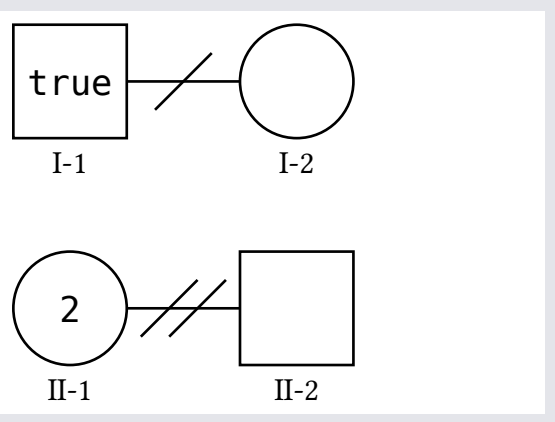
Default: `false`

divorced `bool` or `int`

Whether to draw a cross indicating divorce through the union.

Can be `true`, `false`, `0`, `1`, or `2`. If it is `true` or `1`, then one cross will be drawn. If it is `2`, then two crosses will be drawn.

```
pedigree(
  {
    individual(1, 1, "male", in-label:
`true`)
    individual(1, 2, "female")
    individual(2, 1, "female", in-label:
`2`)
    individual(2, 2, "male")
    union("i1-1", "i1-2", divorced:
true)
    union("i2-1", "i2-2", divorced: 2)
  }
)
```

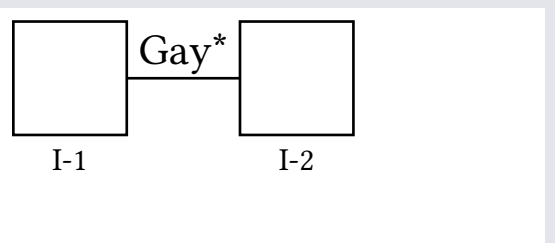


Default: `false`

label `content` or `none`

The label to place above the union line.

```
pedigree(
  {
    individual(1, 1, "male")
    individual(1, 2, "male")
    union("i1-1", "i1-2", label:
[Gay\*])
  }
)
```

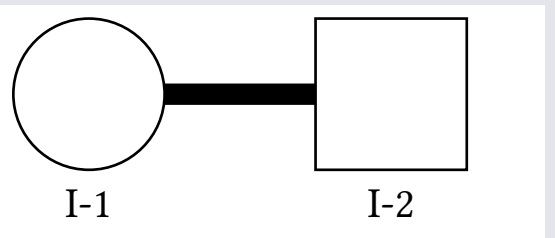


Default: `none`

style `dictionary`

The CeTZ style to give to the union line of this unions. Must be a dictionary.

```
pedigree(
  {
    individual(1, 1, "female")
    individual(1, 2, "male")
    union("i1-1", "i1-2", style:
(thickness: 8pt))
  }
)
```

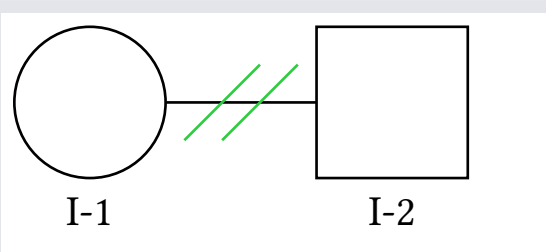


Default: `(:)`

divorced-style `dictionary`

The CeTZ style to give to the separation indicators of this unions. Must be a dictionary.

```
pedigree(
  {
    individual(1, 1, "female")
    individual(1, 2, "male")
    union("i1-1", "i1-2", divorced: 2,
divorced-style: (paint: green))
  }
)
```



Default: `(:)`